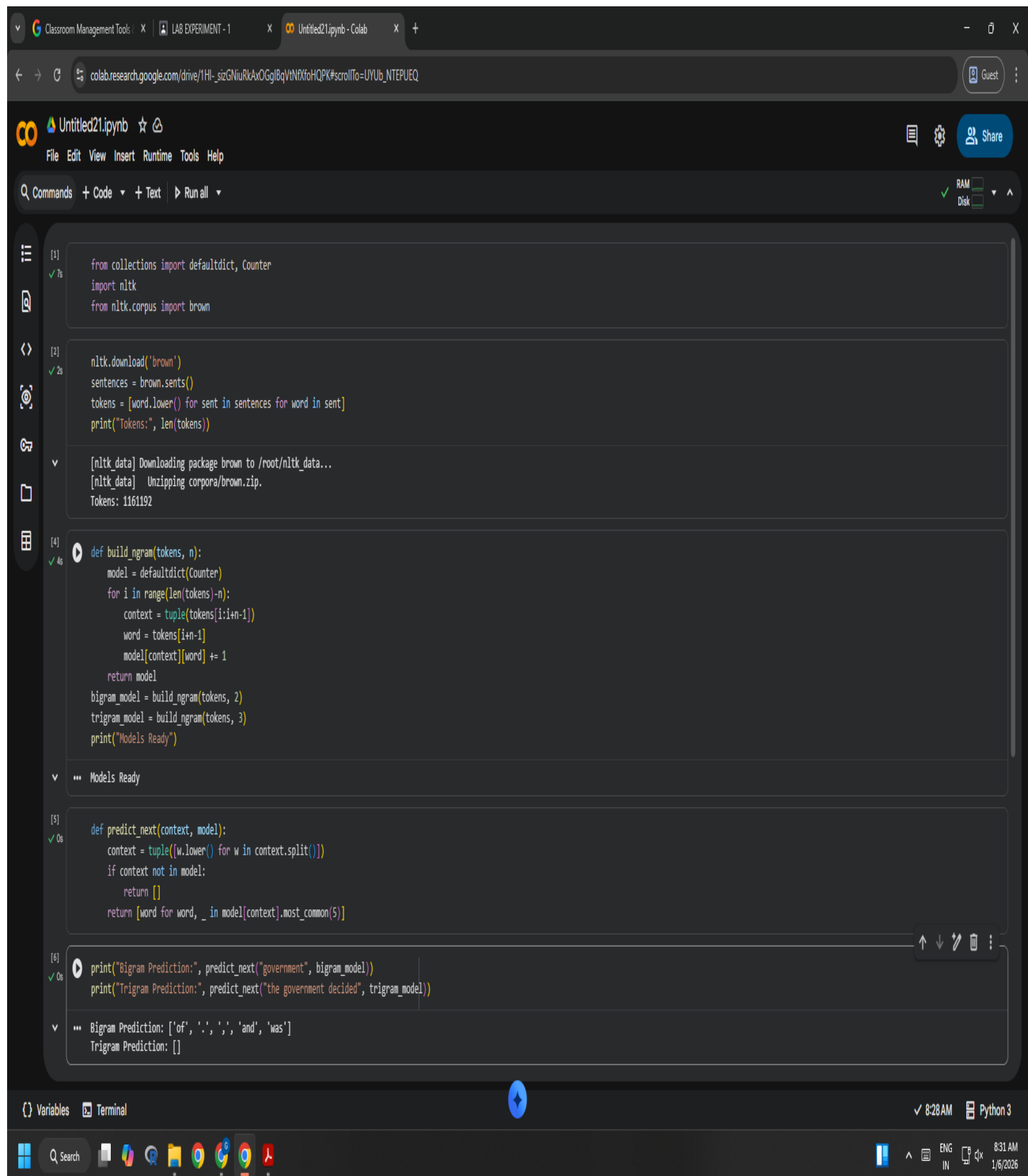


Text Generation using N-gram Language model



The screenshot displays a Google Colab notebook interface with the following components:

- Browser Tabs:** Classroom Management Tools, LAB EXPERIMENT - 1, and the active tab, 'Untitled21.ipynb - Colab'.
- URL:** colab.research.google.com/drive/1H-szGNuRkAvOGgJbQvNRXfoHQPK#scrollTo=UYUb_NTEPUQ
- Notebook Title:** Untitled21.ipynb
- Menu Bar:** File, Edit, View, Insert, Runtime, Tools, Help.
- Command Bar:** Search (Q), Commands, + Code, + Text, Run all.
- Code Cells:**
 - Cell [1]:** Imports required libraries: `from collections import defaultdict, Counter`, `import nltk`, and `from nltk.corpus import brown`.
 - Cell [2]:** Downloads the 'brown' corpus and processes it into tokens: `nltk.download('brown')`, `sentences = brown.sents()`, `tokens = [word.lower() for sent in sentences for word in sent]`, and `print("Tokens:", len(tokens))`. The output shows 1161192 tokens.
 - Cell [4]:** Defines a `build_ngram` function to create n-gram models. It iterates through the tokens to build a model for a given n. It also creates bigram and trigram models: `bigram_model = build_ngram(tokens, 2)` and `trigram_model = build_ngram(tokens, 3)`. The output is 'Models Ready'.
 - Cell [5]:** Defines a `predict_next` function that takes a context and a model, splits the context into words, and predicts the next word based on the most common words in the model's context. It returns an empty list if the context is not in the model.
 - Cell [6]:** Tests the models with example contexts: `print("Bigram Prediction:", predict_next("government", bigram_model))` and `print("Trigram Prediction:", predict_next("the government decided", trigram_model))`. The output shows the Bigram Prediction as `['of', '.', ',', 'and', 'was']` and the Trigram Prediction as an empty list.
- Bottom Bar:** Includes 'Variables' and 'Terminal' tabs, a status bar showing '8:28 AM' and 'Python 3', and a Windows taskbar at the very bottom with various application icons.

231801093
EX 1a